Lab 7 – Fuzzing for Input Validation Bugs

Name: Akash M P **SRN:** PES1UG24CS040

1. Objective

The objective of this lab is to perform property-based fuzz testing on an elevator dispatch program (elevator_dispatch.py) using Python 3.12, pytest, and Hypothesis. The aim is to identify edge-case bugs and unhandled exceptions (such as None inputs, empty strings, invalid delimiters, and zero division), and harden the code by raising descriptive ValueError exceptions rather than allowing unexpected crashes.

2. Introduction to Fuzz Testing

Fuzz testing (fuzzing) is an automated testing technique where a large number of random, unexpected, or malformed inputs are passed into a program to check if it crashes or fails. Unlike manual testing which verifies anticipated values, fuzzing tests wide input domains to find unanticipated edge cases.

Hypothesis is a property-based testing framework for Python. Instead of testing static hand-written values, developers define general properties that code must satisfy across generated strategies (such as st.text() and st.none()). When Hypothesis discovers a failing input, it systematically shrinks it to the smallest reproducible counterexample, simplifying debugging.

3. Software Requirements

- **Operating System:** Microsoft Windows
- **Programming Language:** Python 3.12 (Python 3.12.10)
- **Test Runner:** pytest (version 9.1.1)
- **Fuzzing Framework:** Hypothesis (version 6.170.14)

4. Functions Tested

The elevator dispatch module contains three core functions:

- **parse_request(text):** Parses an elevator request in FROM->TO format (e.g., '5->9') and returns start and destination floor numbers as an integer tuple (start, end).
- **total_travel(floors_csv):** Parses comma-separated visited floors (e.g., '1,5,3,9') and computes cumulative distance travelled between consecutive floors (|5-1| + |3-5| + |9-3| = 12).
- **load_percent(text):** Parses load and capacity in 'LOAD of CAPACITY' format (e.g., '450 of 630') and computes load percentage using integer division (load * 100 // capacity).

5. Procedure

1. Ran original elevator_dispatch.py with standard valid inputs to check baseline behavior.
2. Created property-based fuzz tests in test_elevator_dispatch.py using Hypothesis strategies (@given, @example).
3. Executed pytest and observed that all 4 tests failed due to unhandled exceptions.
4. Analyzed minimal failing counterexamples found by Hypothesis to identify root causes.
5. Added input validation, type checking, and boundary conditions to raise ValueError on invalid inputs.
6. Re-ran application with valid inputs to confirm that correct calculations remain unaffected.
7. Re-executed Hypothesis test suite to verify that all 4 tests pass cleanly without crashes.

6. Bugs Identified

Hypothesis fuzz testing identified three primary bugs in the original code:

- **AttributeError due to None inputs:** The functions called .split() directly without checking for None. When Hypothesis supplied None, an unhandled AttributeError: 'NoneType' object has no attribute 'split' occurred across all three functions.
- **ZeroDivisionError due to zero capacity:** In load_percent(), capacity was passed directly into // int(capacity) without zero checking. Input '100 of 0' triggered an unhandled ZeroDivisionError: integer division or modulo by zero.
- **Missing input validation:**
  - Missing delimiters (e.g. '') caused unpacking errors (ValueError: not enough values to unpack).
  - Extra delimiters (e.g. '5->9->10') caused unpacking errors (ValueError: too many values to unpack).
  - Empty or non-numeric tokens (e.g. '1,,3') crashed int() parsing inside list comprehension.
  - Negative floor numbers and loads exceeding capacity were accepted without validation.

7. Corrections Made

The following fixes were implemented in elevator_dispatch.py:

- **parse_request(text):**
  1. Added isinstance(text, str) check; raises ValueError('Input must be a string') if input is None or non-string.
  2. Checked that text.split('->') produces exactly 2 elements (len(parts) == 2).
  3. Converted floor values to integers inside try...except ValueError.
  4. Ensured floors are not negative (start < 0 or end < 0).
- **total_travel(floors_csv):**
  1. Checked that floors_csv is a string and not empty (not floors_csv.strip()).
  2. Stripped and safely parsed floor values in try...except ValueError.
  3. Added check that all floors are non-negative (any(f < 0 for f in floors)).
- **load_percent(text):**
  1. Added string type check and verified split on ' of ' gives 2 elements.
  2. Safely parsed load and capacity as integers.
  3. Added boundary validations: load < 0, capacity <= 0 (prevents ZeroDivisionError), and load > capacity.

8. Results and Screenshots

Comparison of test results before and after fixes:

Test Case / Scenario	Original Implementation	Corrected Implementation
Application with valid inputs	Normal outputs produced	Normal outputs maintained
test_parse_request_no_crash	**FAILED** (AttributeError on None, unpacking)	**PASSED** (Graceful ValueError)
test_total_travel_no_crash	**FAILED** (AttributeError on None, empty tokens)	**PASSED** (Graceful ValueError)
test_load_percent_no_crash	**FAILED** (AttributeError on None, ZeroDivisionError)	**PASSED** (Graceful ValueError)
test_zero_capacity	**FAILED** (ZeroDivisionError on '100 of 0')	**PASSED** (Raises ValueError as expected)
Overall Test Suite	**4 FAILED in initial test run**	**4 PASSED in 0.94s**

Screenshot 1: Original Application Output

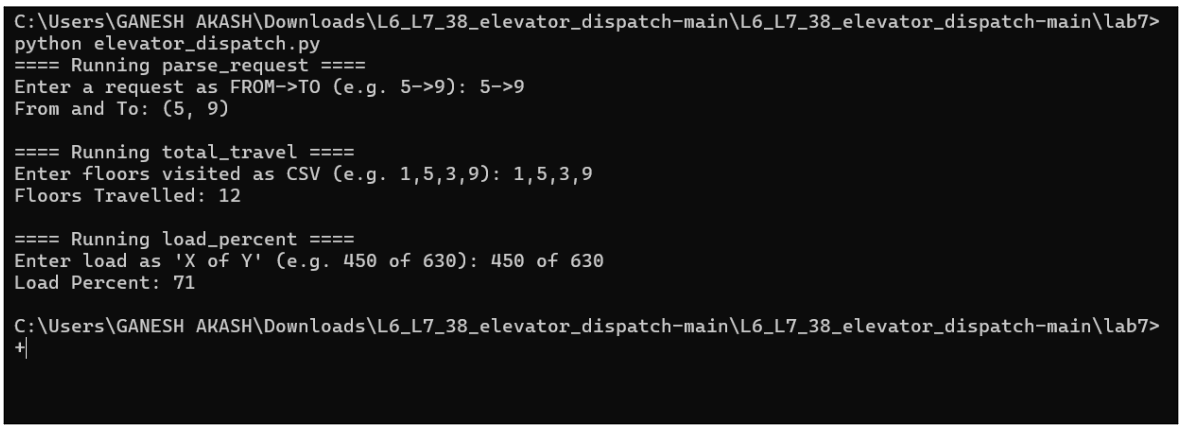


Figure 1: Original application running with valid inputs.

Screenshot 2: Original Hypothesis Fuzz-Testing Failures (4 Failed)

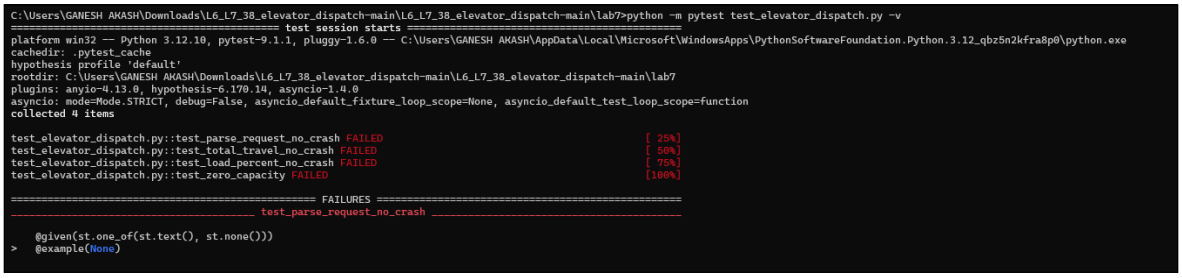


Figure 2: pytest run showing 4 test failures caught by Hypothesis.

Screenshot 3: Corrected Application Output

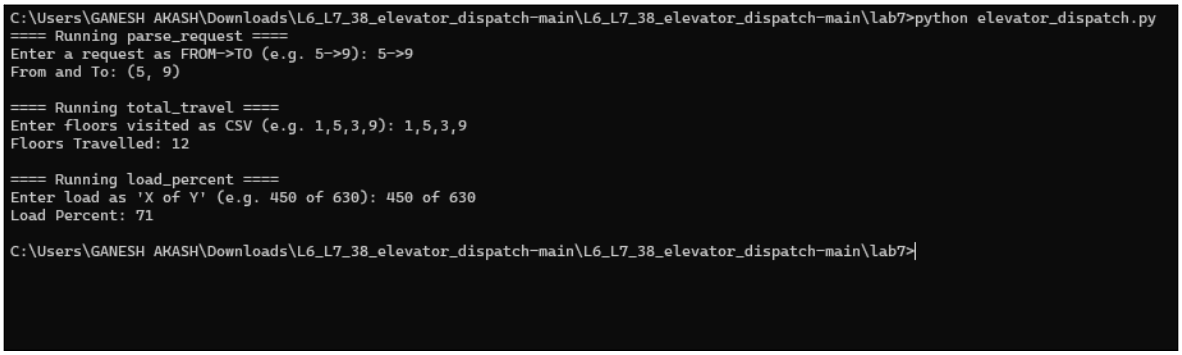


Figure 3: Corrected application running normally on valid inputs.

Screenshot 4: Final Hypothesis Test Results (4 Passed)

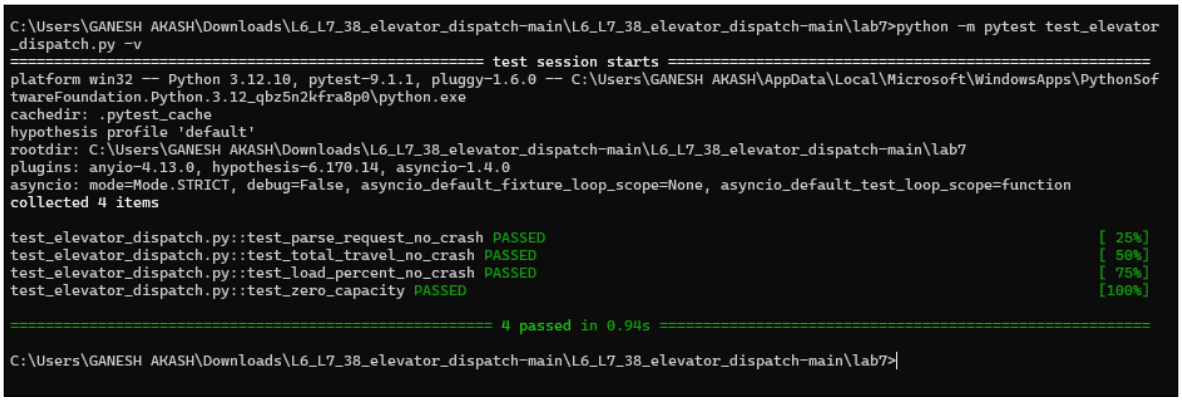


Figure 4: Final pytest run showing all 4 tests passing in 0.94s.

9. Conclusion

Fuzz testing with Hypothesis was very effective in finding edge-case bugs and unexpected crashes that manual testing missed. Property-based tests automatically generated failing inputs such as None values, empty strings, zero capacity, and invalid formats, and simplified them to minimal reproducible examples.

By adding type checking, format validation, range checking, and raising descriptive ValueError exceptions, the elevator dispatch module was made robust and fault-tolerant against unexpected inputs.